UNIVERSITÀ DEGLI STUDI FIRENZE

Da un secolo, oltre.

HR EXCELLENCE IN RESEARCH

02.2: Network Function Virtualization (NFV)

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Foundational Technology Enablers

Objective: To introduce the foundational concepts and technologies that enable the flexible, agile, and intelligent network infrastructure required for 6G and beyond.

Lesson 2.1: Software-Defined Networking (SDN)

- **Duration:** 4 Hours
- **Focus:** Understanding the core principles of SDN, including the decoupling of control and data planes, the role of the centralized controller, and how it enables programmatic network management and agility.

Lesson 2.2: Network Functions Virtualization (NFV)

- **Duration:** 4 Hours
- **Focus:** Exploring how network functions are virtualized and run as software on commodity hardware, leading to cost reduction, increased flexibility, and faster service deployment. We'll also examine the synergy between SDN and NFV.

Lesson 2.3: The Cloud-to-Edge Continuum

- **Duration:** 2 Hours
- **Focus:** Examining the distributed computing paradigm that underpins 6G, including the roles of centralized cloud computing and highly distributed edge computing, and how they seamlessly integrate to form a powerful, intelligent infrastructure fabric.

The NFV Paradigm

The Economic Driver: CapEx vs. OpEx and "Middlebox Hell"

Before NFV, telecommunication providers and large enterprises faced significant economic and operational challenges due to a hardware-centric approach, often referred to as "Middlebox Hell."

The Traditional Telco/Enterprise Model: Hardware-Dominated

- This model was historically dominated by **high Capital Expenditures (CapEx)**.
- Organizations had to purchase and deploy numerous **specialized, proprietary hardware appliances** for every network function:
  - Dedicated **firewalls**
  - Purpose-built **routers**
  - Specialized **load balancers**
  - Intrusion detection systems, WAN optimizers, and many more "middleboxes."
- Each of these appliances often came from a different vendor, requiring its own management interface and operational silo.

The Economic Driver: CapEx vs. OpEx and "Middlebox Hell"

The Problem with this Model: A Cycle of Inefficiency

1. **High Upfront Cost & Risk:**
   - Acquiring specialized hardware requires significant initial investment (CapEx).
   - This investment carries inherent risk, as the hardware might become obsolete or inadequate for future needs before its depreciation period ends.
2. **High Operational Expenditures (OpEx):**
   - Each physical box requires power, cooling, and dedicated physical space in data centers or central offices.
   - More critically, it demands specialized staff with expertise in each vendor's proprietary hardware and software, leading to increased labor costs for configuration, maintenance, and troubleshooting.

The Economic Driver: CapEx vs. OpEx and "Middlebox Hell"

3. **Stranded Capacity:**
   - Hardware is typically purchased and provisioned for **peak load** requirements to ensure service availability during busy periods.
   - This often results in **massive underutilization** of resources during off-peak times, leading to "stranded capacity" – expensive hardware sitting idle.

4. **Inflexibility & Slow Time-to-Market:**
   - Launching a new network service or deploying a new network function could take anywhere from **6 to 12 months**. This lengthy process involved:
     - Procurement of new hardware (long lead times).
     - Physical installation, cabling, and power-up.
     - Manual configuration and integration.
   - This inflexibility severely hampered innovation and responsiveness to market demands.

The NFV Principle: Decoupling Software from Specialized Hardware

Building on the economic challenges of traditional hardware-centric networks, NFV introduces a fundamental shift in how network functions are deployed and managed.

The Core Insight: Software is the Value

- The fundamental insight behind NFV is that the true value of a network appliance lies in its **software logic** and the function it performs, **not** in the proprietary, custom hardware it runs on.

The NFV Solution: Virtualization and Commodity Hardware

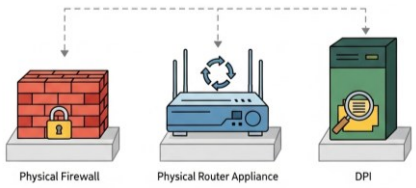
NFV offers a revolutionary solution by:

1. **Decoupling the network function software from the proprietary hardware:** This means functions like firewalls, routers, or load balancers are no longer tied to specific physical boxes.
2. **Virtualizing it, creating a Virtualized Network Function (VNF):** The software for these network functions is packaged as a VNF, which can run in a virtualized environment.
3. **Deploying it on generic, Commercial Off-The-Shelf (COTS) servers:** These VNFs run on standard, high-volume servers, storage, and networking hardware, typically within a standard cloud infrastructure (e.g., using hypervisors like KVM, VMware, or container runtimes like Docker).

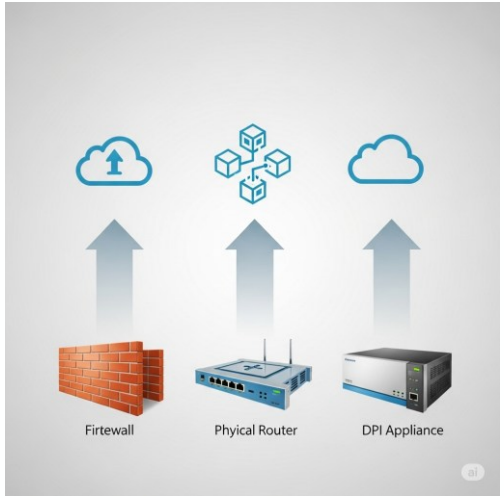
This shift allows network operators to leverage the economies of scale and flexibility of IT virtualization, bringing cloud-like agility to network deployments.

Virtualization and Commodity Hardware

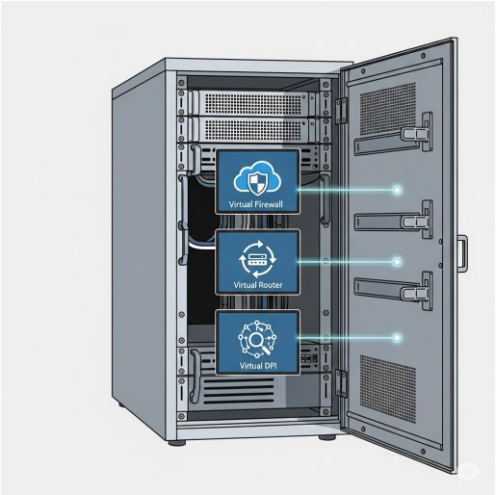
(1) Physical Appliances



(2) Virtualized Functions



(3) COTS Server Deployment



The ETSI Architecture

The ETSI NFV Architectural Framework: An Overview

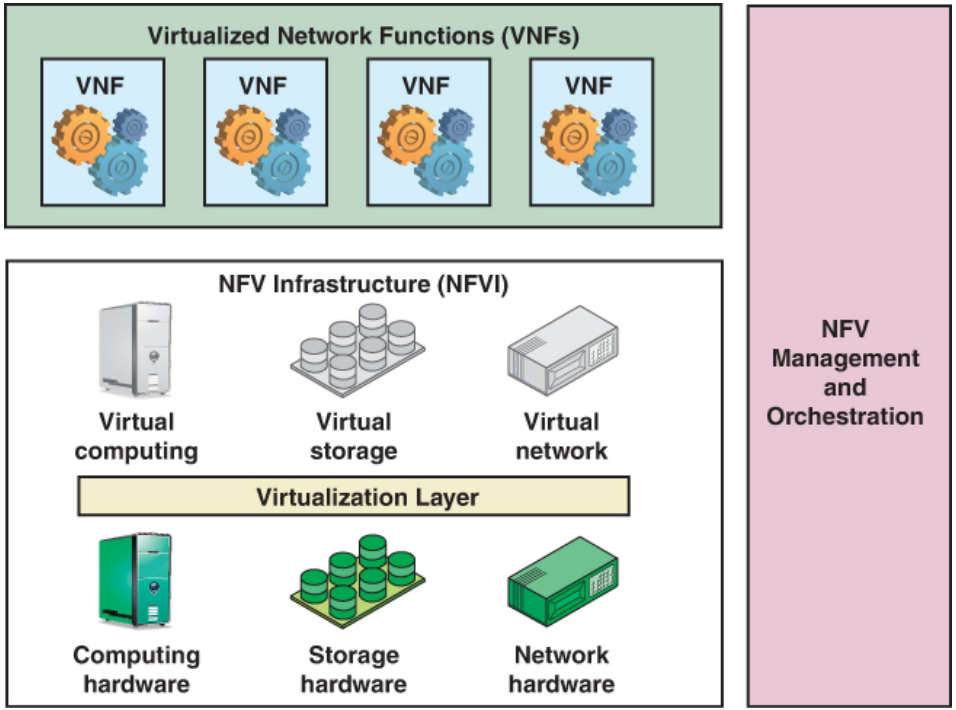
To ensure interoperability and provide a common understanding for the development and deployment of NFV, the European Telecommunications Standards Institute (ETSI) has defined a comprehensive architectural framework. This framework provides a high-level overview of the key components and their interactions within an NFV environment.

The Main Architectural Blocks:

The ETSI NFV Architectural Framework is typically depicted with three top-level functional blocks:

1. **Virtualized Network Functions (VNFs):** These are the software implementations of network functions that run on the NFV Infrastructure.
2. **NFV Infrastructure (NFVI):** This comprises the hardware resources (compute, storage, network) and the virtualization layer (e.g., hypervisors) on which VNFs are deployed.
3. **NFV Management and Orchestration (MANO):** This is the functional block responsible for managing and orchestrating all resources in the NFV environment, including VNFs and the NFVI.

ETSI NFV Framework

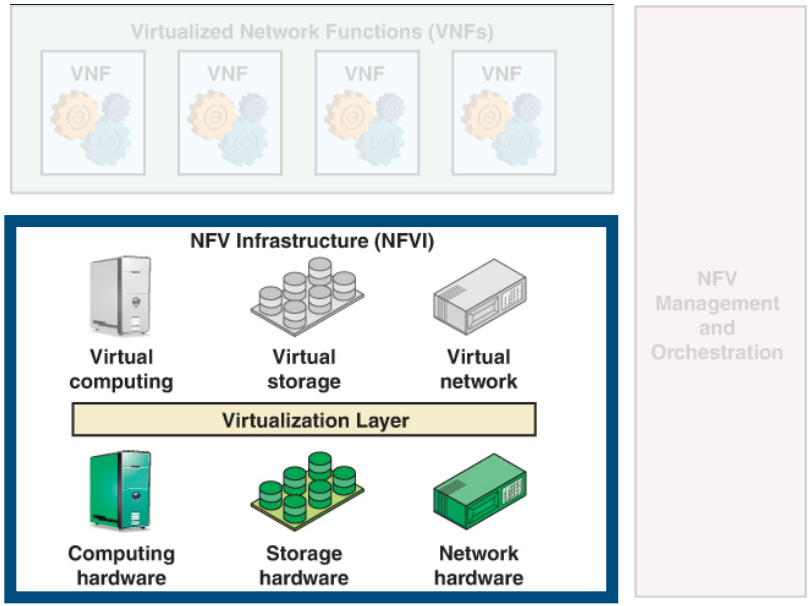


Architecture Deep Dive 1: The NFV Infrastructure (NFVI)

The NFV Infrastructure (NFVI) is the bedrock upon which all virtualized network functions operate. It provides the necessary compute, storage, and networking resources in a virtualized manner.

Purpose: Providing Virtualized Resources for VNFs

- The primary purpose of the NFVI is to provide the **virtualized resources** (compute, storage, and network connectivity) that are needed by the **Virtualized Network Functions (VNFs)**.
- It abstracts the underlying hardware, allowing VNFs to run independently of specific physical configurations.



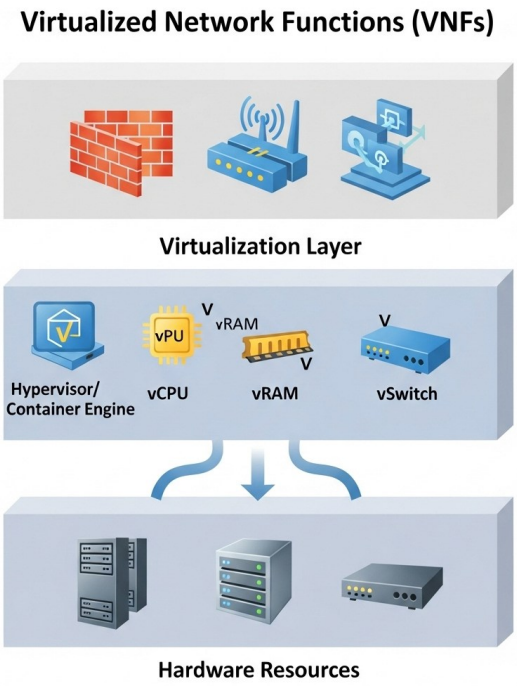
NFVI Components

The NFVI is composed of two main sets of components:

1. **Hardware Resources:**
   - **COTS (Commercial Off-The-Shelf) Compute:** Standard servers, typically based on x86 or ARM architectures, providing the processing power (vCPU).
   - **Storage:** Common storage solutions such as SAN (Storage Area Network), NAS (Network-Attached Storage), or DAS (Direct-Attached Storage) that provide the virtualized disk space (vDisk).
   - **Networking:** Standard network switches and routers that provide the physical connectivity for the virtualized environment.
2. **Virtualization Layer:**
   - This is the core software that enables the abstraction of hardware resources.
   - **Hypervisor (e.g., KVM, VMware ESXi, Xen):** For deploying VNFs as **Virtual Machines (VMs)**. The hypervisor creates and manages the virtualized hardware (vCPU, vRAM, vNICs, vSwitch) that VMs utilize.
   - **Container Engine (e.g., Docker, Kubernetes):** For deploying VNFs as **containers**. Containers provide a lighter-weight virtualization by sharing the host OS kernel.
   - **Role:** This layer manages the allocation and isolation of virtual resources (like virtual CPUs, virtual RAM, and virtual switches) to the VNFs.

NFV Infrastructure

The NFVI is essentially a private or public IaaS (Infrastructure as a Service) cloud, but it's often specifically optimized for networking workloads, requiring high performance, low latency, and robust network connectivity for the VNFs it hosts.

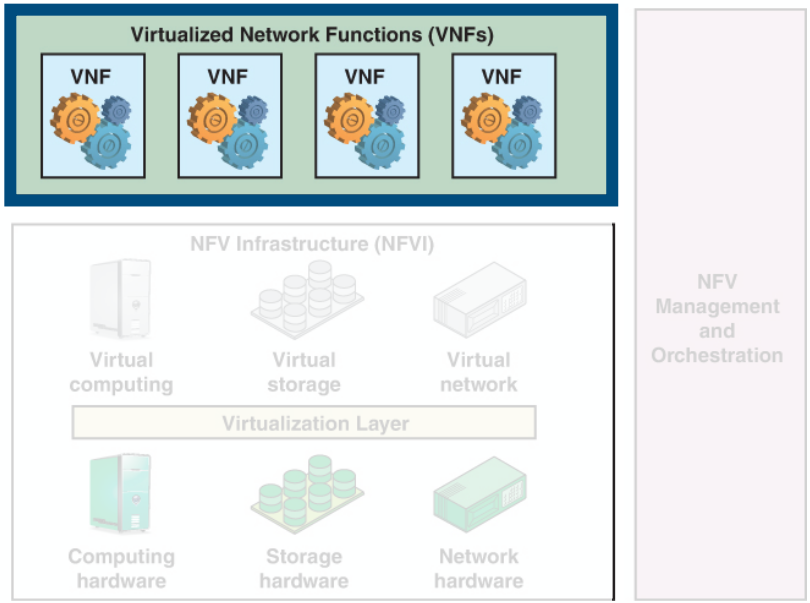


Architecture Deep Dive 2: The Virtualized Network Function (VNF)

Having explored the foundational NFV Infrastructure, let's now delve into the core software components that run on it: the Virtualized Network Functions (VNFs).

Definition: A Software Implementation of a Network Function

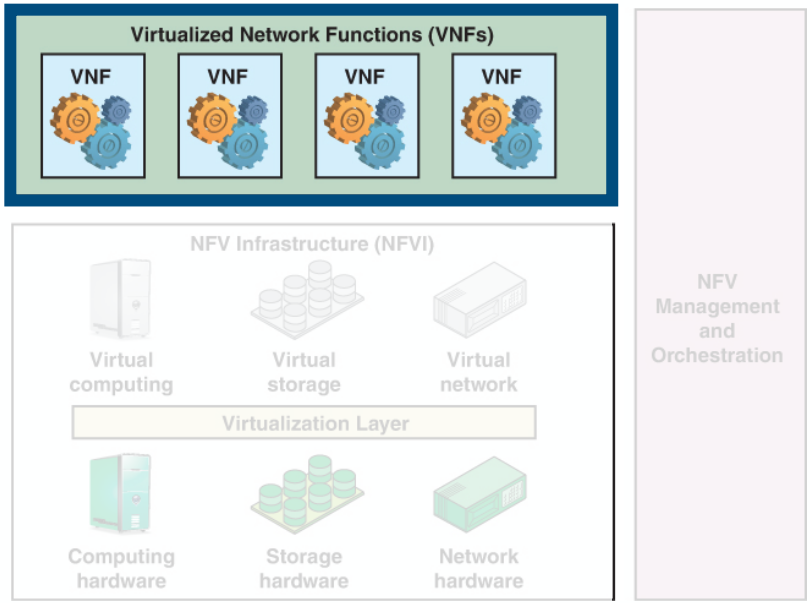
- A **Virtualized Network Function (VNF)** is a software implementation of a traditional network function (e.g., a firewall, router, load balancer, or deep packet inspection engine) that runs on standard, commodity hardware instead of proprietary, dedicated appliances.



Architecture Deep Dive 2: The Virtualized Network Function (VNF)

Structure: Composed of VNF Components (VNFCs)

- A VNF is typically composed of one or more **VNF Components (VNFCs)**.
- Each **VNFC** represents a logical part of the VNF's functionality and is usually mapped to a single **Virtual Machine (VM)** or **container**. This modularity allows for flexible scaling and deployment of different VNF parts.
  - *Example:* A virtual firewall VNF might have one VNFC for packet filtering, another for session management, and another for logging, each running in its own VM or container.



The VNF Descriptor (VNFD): The "User Manual" for the VNF

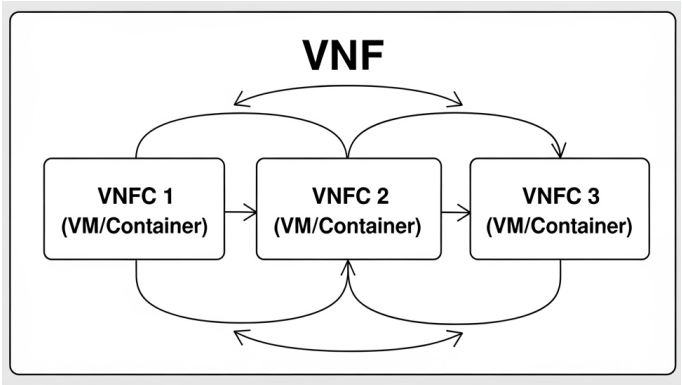
The **VNF Descriptor (VNFD)** is a **critical artifact** in the NFV ecosystem. It's a template file, typically written in a human-readable format like **YAML** (or sometimes XML), that acts as the "user manual" or blueprint for deploying and managing a specific VNF.

It provides all the necessary information for the NFV Management and Orchestration (MANO) system to instantiate, configure, and scale the VNF.

A VNFD specifies:

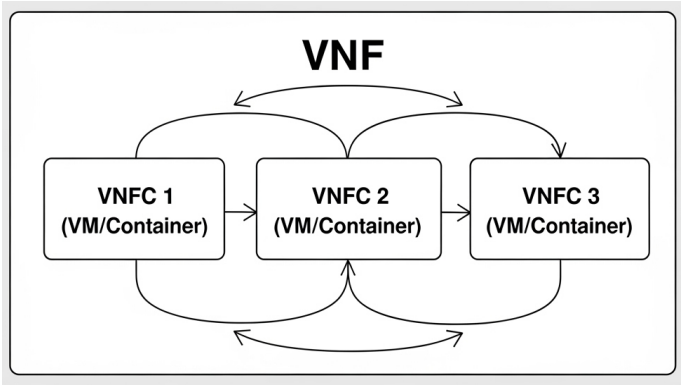
- **Resource Requirements:** The virtual compute (vCPU), virtual memory (vRAM), and virtual storage (vDisk) needed for each VNFC.
- **Software Image Locations:** Where to find the software images (e.g., VM images, container images) for each VNFC.
- **Connectivity Requirements:** How the VNFCs within the VNF should be interconnected, and how the VNF should connect to external networks.
- **Key Performance Indicators (KPIs) and Scaling Policies:** Metrics to monitor the VNF's performance and rules for how and when the VNF should scale up or down (e.g., add more VNFC instances if CPU utilization exceeds 80%).

An example of VNF Descriptor




```yaml
# Simplified VNF Descriptor (VNFD) Snippet
vnf_name: my_virtual_firewall
vnf_version: 1.0
vnf_provider: AcmeNetworks

vnf_components:
  - id: firewall_engine_vnfc
    image: "firewall-engine-v1.0.qcow2"
    resource_requirements:
      vcpus: 4
      vram: 8GB
      vdisk: 50GB
    interfaces:
      - name: mgmt
        type: management
      - name: data_in
        type: data
      - name: data_out
        type: data
```

An example of VNF Descriptor



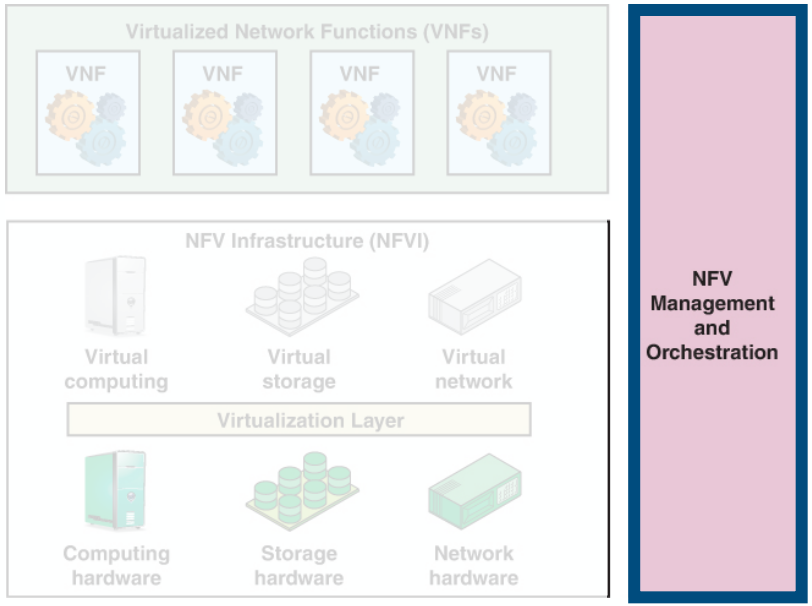

```
- id: logging_vnfc
    image: "logging-service-v1.0.docker"
    resource_requirements:
      vcpus: 2
      vram: 4GB
      vdisk: 100GB
    interfaces:
      - name: mgmt
        type: management
      - name: log_data
        type: internal

scaling_policies:
  - type: auto_scale_out_in
    criteria: cpu_utilization > 80%
    action:
add_vnfc_instance(firewall_engine_vnfc)
```

Architecture Deep Dive 3: The NFV MANO

The NFV Infrastructure (NFVI) provides the resources, and VNFs are the virtualized functions. But what orchestrates and manages them all? That's the role of **NFV MANO**.

MANO: Management and Orchestration

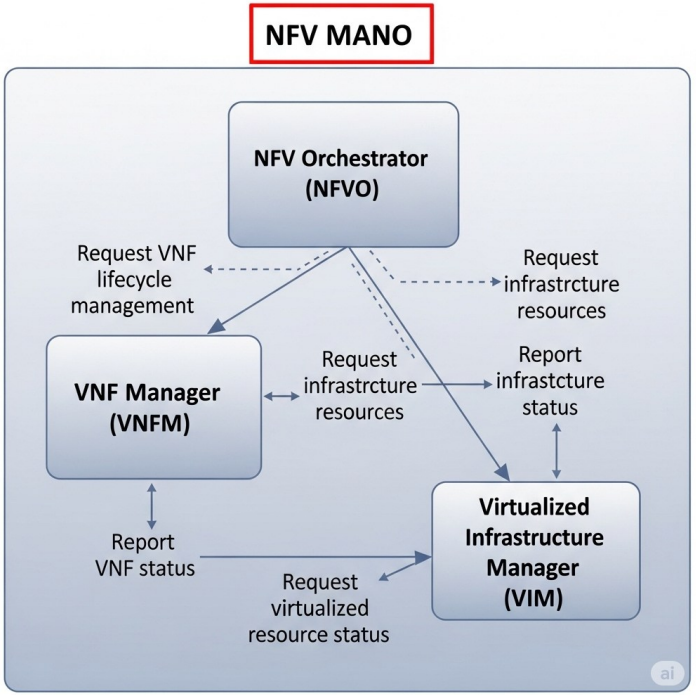
- **MANO** stands for **Management and Orchestration**.
- This is the comprehensive set of functions and components that **automates the entire lifecycle** of VNFs and the NFVI.
- It's the intelligence that brings the NFV vision to life, enabling dynamic resource allocation, VNF deployment, scaling, and overall operational efficiency.



Architecture Deep Dive 3: The NFV MANO

The ETSI NFV Architectural Framework defines MANO as being composed of three primary logical blocks, which work together to achieve end-to-end management and orchestration:

1. **NFV Orchestrator (NFVO):**
   - The highest-level orchestrator.
   - Responsible for orchestrating network services (which are composed of multiple VNFs) and managing their lifecycle across the entire NFV infrastructure.
2. **VNF Manager (VNFM):**
   - Responsible for the lifecycle management of individual VNFs.
   - This includes instantiation, scaling, healing, and termination of VNFs.
3. **Virtualized Infrastructure Manager (VIM):**
   - Responsible for controlling and managing the NFVI resources (compute, storage, and network).
   - It allocates and manages the virtual resources provided by the hypervisors or container engines.



MANO Component 1: The VIM (Virtualized Infrastructure Manager)

The VIM is the lowest-level component within the NFV MANO stack, directly interacting with and managing the NFV Infrastructure (NFVI).

Responsibility: Managing NFVI Resources

- The **Virtualized Infrastructure Manager (VIM)** is primarily responsible for **controlling and managing the NFVI resources**. This includes the underlying compute, storage, and networking hardware, as well as the virtualization layer (hypervisors or container engines).

The VIM performs several critical functions to enable the operation of VNFs:

- **Resource Allocation and Release:** It dynamically allocates and releases **virtual compute** (vCPUs, vRAM), **virtual storage** (vDisks), and **virtual networking** resources (e.g., virtual switches, virtual NICs) to VNFs as requested by higher-level MANO components.
- **Performance and Fault Information Collection:** It continuously collects **performance metrics** (e.g., CPU utilization, memory usage, network throughput) and **fault information** (e.g., hardware failures, hypervisor issues) from the underlying infrastructure. This data is crucial for monitoring, troubleshooting, and making intelligent resource management decisions.
- **Virtual Resource Management:** It manages the lifecycle of virtual resources, ensuring proper isolation and efficient utilization.

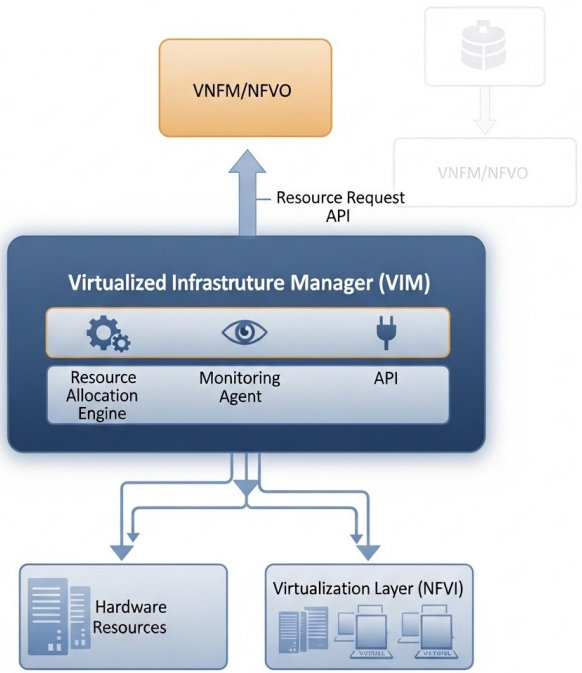
MANO Component 1: The VIM (Virtualized Infrastructure Manager)

Standard Examples:

- Well-known examples of VIM implementations include **OpenStack** and **VMware vCenter**. These platforms provide the necessary capabilities to manage large-scale virtualized environments.

Interface: The Resource Request API

- The VIM exposes an **API (Application Programming Interface)** that the **VNF Manager (VNFM)** and the **NFV Orchestrator (NFVO)** use to request and manage virtual resources. This northbound interface allows higher-level components to abstract away the specifics of the underlying hardware.



MANO Component 2: The VNFM (VNF Manager)

The VNF Manager (VNFM) is a crucial component within the NFV MANO stack, directly responsible for the lifecycle management of individual Virtualized Network Functions (VNFs).

Responsibility: Managing the Entire Lifecycle of a VNF Instance

- The **VNF Manager (VNFM)** is dedicated to overseeing the complete lifecycle of a **VNF instance**, from its initial creation to its eventual termination. It ensures that VNFs are deployed, scaled, and maintained according according to their defined requirements.

Key Functions:

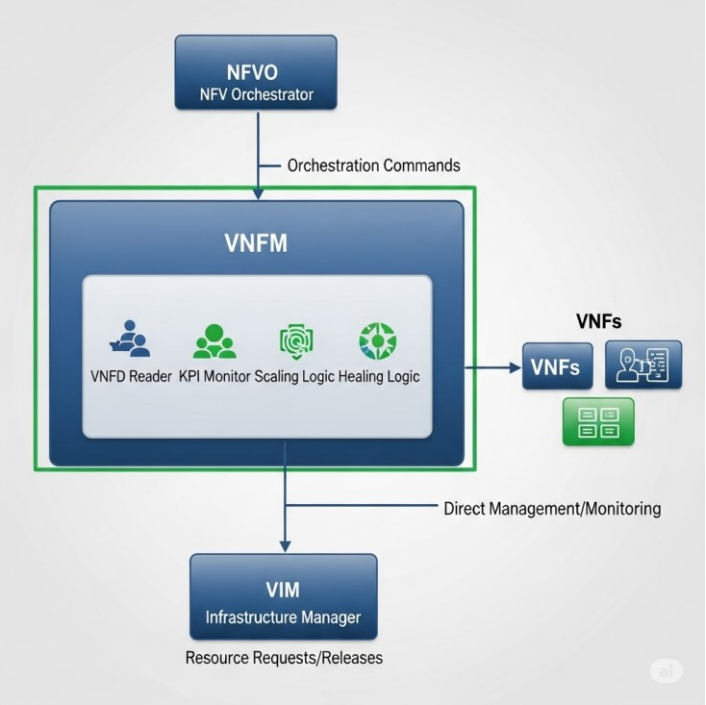
The VNFM performs several vital functions:

- **Instantiation:**
  - Reads the **VNF Descriptor (VNFD)**, which acts as the blueprint for the VNF.
  - **Coordinates with the VIM** (Virtualized Infrastructure Manager) to request and allocate the necessary virtual compute, storage, and networking resources to create a new VNF instance.
  - Deploys the VNF software image onto the allocated resources.

MANO Component 2: The VNFM (VNF Manager)

- **Scaling:**
  - **Monitors the VNF's Key Performance Indicators (KPIs)** (e.g., CPU utilization, throughput, connection count) as defined in the VNFD.
  - Based on pre-defined **scaling policies** (also from the VNFD) and current performance, it triggers **scaling actions**:
    - **Scale Out/In:** Adding or removing VNFC instances (e.g., deploying more virtual firewall instances to handle increased traffic).
    - **Scale Up/Down:** Increasing or decreasing the resources allocated to an existing VNFC instance (e.g., giving a VM more vCPUs or vRAM).
- **Healing:**
  - **Detects failures** of individual VNF Components (VNFCs) or the VNF instance itself.
  - Automatically initiates recovery actions, such as **recreating failed VNFCs** on healthy infrastructure, migrating VNFs, or restarting processes to ensure service continuity.
- **Termination:**
  - Gracefully **shuts down a VNF instance** when it is no longer needed.
  - Ensures that all associated virtual resources are **released back to the VIM**, making them available for other VNFs.

The VNFM (VNF Manager)



MANO Component 3: The NFVO (NFV Orchestrator)

The NFV Orchestrator (NFVO) sits at the top of the MANO hierarchy, providing the overarching intelligence to manage and orchestrate complete network services.

Responsibility: Orchestrating at the Network Service Level

- The **NFV Orchestrator (NFVO)** is responsible for orchestrating entire **Network Services**.
- A **Network Service** is a logical construct that represents an end-to-end service provided to a customer. It is typically composed of a **collection of VNFs** (e.g., a virtual firewall, a virtual router, and a virtual load balancer chained together) and the **connections between them**.

Key Functions:

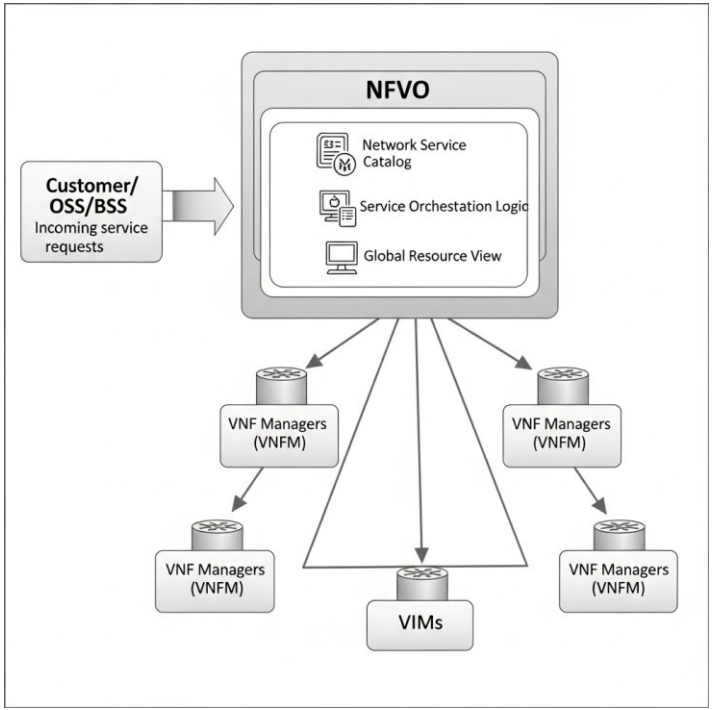
The NFVO performs critical functions for end-to-end service delivery:

- **Onboarding:**
  - Manages the **catalog of available Network Services and VNFs**. This includes receiving and validating Network Service Descriptors (NSDs) and VNF Descriptors (VNFDs).
  - Makes these service definitions available for deployment.

MANO Component 3: The NFVO (NFV Orchestrator)

- **End-to-End Orchestration:**
  - This is the core function. When a request comes to deploy a new network service, the NFVO takes the NSD as input.
  - It then **coordinates with multiple VNF Managers (VNFMs)** to instantiate and configure the individual VNFs that compose the service.
  - It also interacts with the **Virtualized Infrastructure Manager (VIM)** (often indirectly via the VNFM) to ensure the necessary virtual resources are allocated across the NFVI.
  - It handles the **interconnection** of these VNFs to form the complete service chain.
- **Global Resource Management:**
  - The NFVO maintains a **global view of resources** available across potentially multiple VIMs or NFVI-PoPs (Points of Presence).
  - This allows it to make intelligent decisions about where to deploy VNFs and network services to optimize performance, resource utilization, and cost.

The NFVO (NFV Orchestrator)



The VNF Lifecycle

The **VNF lifecycle** is the set of phases, from initial deployment to decommissioning, managed automatically by the **NFV MANO framework**. This agile approach transforms network functions into dynamic software, offering greater efficiency and responsiveness than traditional hardware management.

State Diagram

The lifecycle of a VNF follows distinct, interconnected states:

- **On-boarding:** The initial phase where the **VNF software package**—including the **VNF Descriptor (VNFD)** and software images—is loaded and registered, making it available for deployment.
- **Instantiation:** A new **VNF instance is created**. The VNF Manager coordinates with the VIM to **allocate virtual resources** (vCPU, vRAM, vDisk) on the NFVI and launch the VNF components.

The VNF Lifecycle

Scaling: The dynamic adjustment of a VNF's resources or instances based on workload.

- **Scale Out/In:** Increases or decreases the number of VNF component instances to handle traffic changes.
- **Scale Up/Down:** Increases or decreases the resources (e.g., vCPUs, vRAM) of an existing VNF instance.

Monitoring/Healing: The VNF Manager **continuously monitors** the VNF's health and performance. If a failure is detected, the system can automatically trigger **healing actions**, like restarting a component, to enhance reliability without manual intervention.

Termination: The final phase where the VNF is **deactivated and removed**. The VNF Manager gracefully shuts down the instance, and all **associated virtual resources** are **released back to the VIM**, making them available for other services.

Service Delivery, Performance, and Cloud-Native Evolution

Concept in Focus: Service Function Chaining (SFC)

Service Function Chaining (SFC) is a cornerstone concept in software-defined networks, allowing for unprecedented flexibility in how network services are delivered.

Definition: Steering Traffic Through an Ordered Sequence of VNFs

- **Service Function Chaining (SFC)** is the process of creating an end-to-end **Network Service** by dynamically steering traffic through an **ordered sequence of Virtualized Network Functions (VNFs)**.
- Instead of a packet taking a fixed path through a series of physical appliances, SFC allows its path to be defined and enforced through a chain of software-based functions, regardless of their physical location.

The Power of SFC: Flexible, Customized Service Chains On Demand

- SFC provides immense power and agility by enabling the creation of **flexible, customized service chains on demand**.
- This was virtually impossible with traditional physical appliances, where reconfiguring a service chain meant physically re-cabling or manually reconfiguring multiple devices, a time-consuming and error-prone process.
- With SFC, you can dynamically insert, remove, or reorder network functions (like firewalls, NATs, load balancers, or WAN optimizers) into a traffic flow based on specific application needs, user policies, or real-time network conditions.

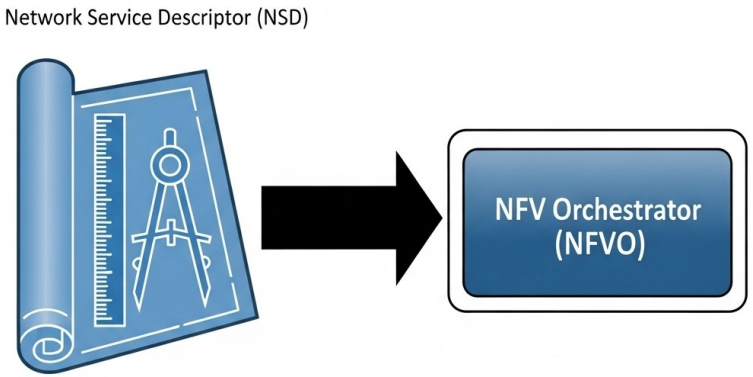
Concept in Focus: Service Function Chaining (SFC)

The Role of SDN: The Key SDN-NFV Synergy

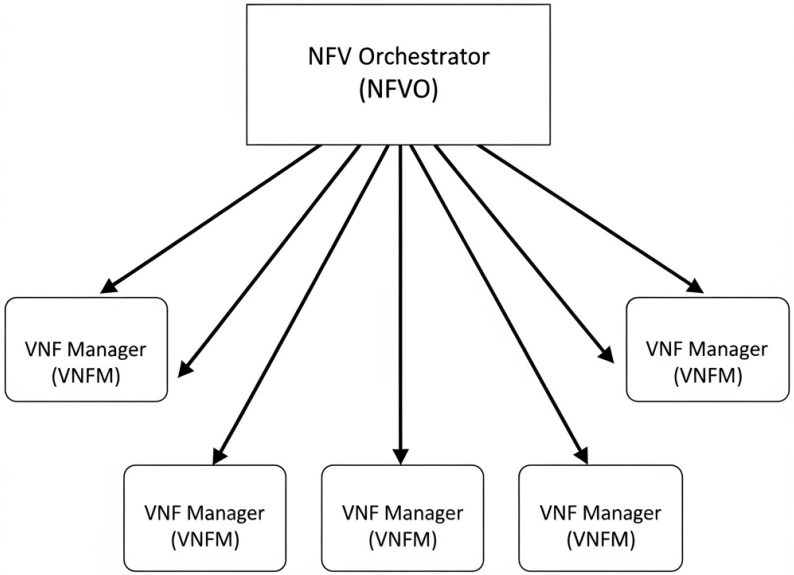
- While **NFV MANO** (specifically the NFV Orchestrator and VNF Managers) is responsible for **instantiating and managing the lifecycle of the individual VNFs** that make up the chain, **SDN plays the critical role of enforcing the traffic steering.**
- An **SDN controller** is the ideal tool to program the underlying network fabric (specifically the **virtual switches (vSwitches) in the NFVI**) to enforce the precise traffic steering required by the service chain.
- This involves installing flow rules in the vSwitches that direct packets from one VNF to the next in the specified sequence, and then out to their final destination.
- **This is the key SDN-NFV synergy:** NFV provides the flexible building blocks (VNFs), and SDN provides the dynamic, programmable "plumbing" to connect and steer traffic through these blocks, creating sophisticated, on-demand network services.

SFC Deployment

Network Service Descriptor sent to the NFV Orchestrator

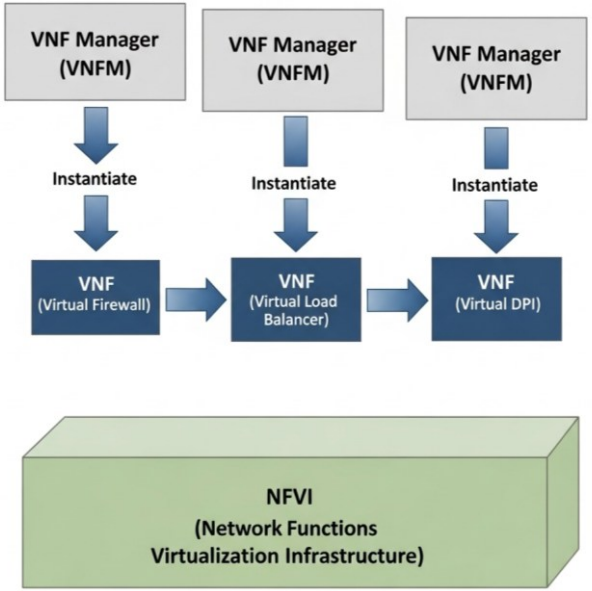


NFVO sends instructions to multiple VNF Managers

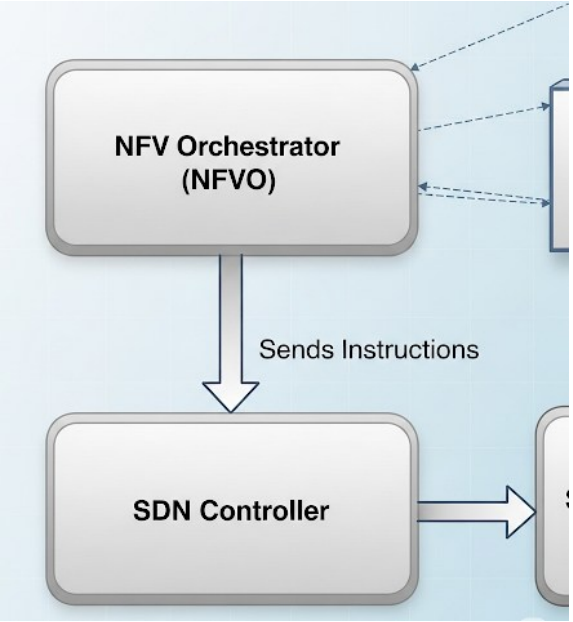


SFC Deployment

Each VNFM instantiates a VNF on the NFVI

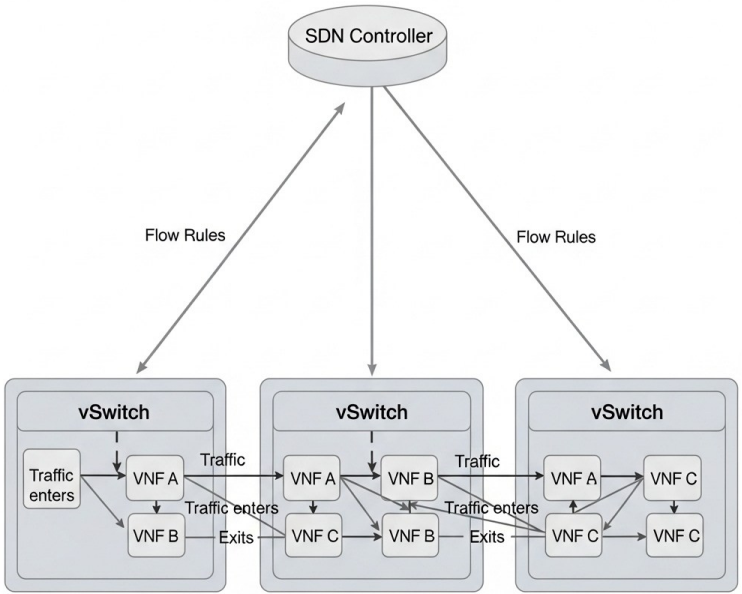


NFVO instructs an SDN Controller

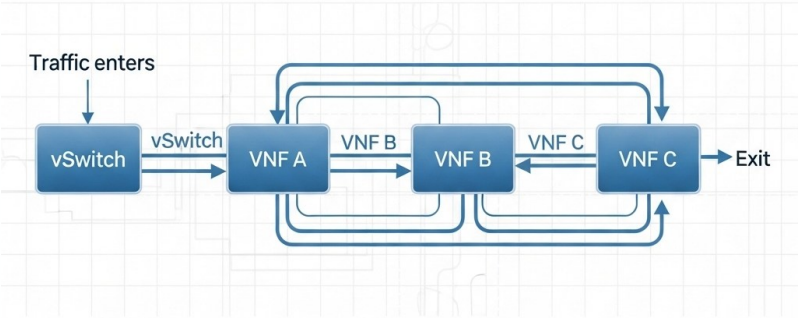


SFC Deployment

SDN Controller programs the vSwitches with flow rules



Packets flowing along the software-defined path



The Challenge of Performance in a Virtual World

While virtualization brings immense flexibility and cost savings, it introduces a significant challenge: how to maintain the high performance traditionally delivered by specialized hardware in a software-defined environment.

The "Performance Tax" of Virtualization

- **Overhead and Latency:** Running network functions as software on general-purpose servers, especially within Virtual Machines (VMs) managed by a hypervisor, introduces inherent overhead and latency.
- **Hypervisor and Virtual Switch:** The hypervisor layer and the virtual switch (vSwitch) that connects VMs to the physical network add processing steps and delays compared to direct interaction with bare-metal hardware.
- **Unacceptable for High-Throughput Functions:** This "performance tax" is often unacceptable for many high-throughput and low-latency network functions, such as a **5G User Plane Function (UPF)**, which must process millions of packets per second with minimal delay.

The Challenge of Performance in a Virtual World

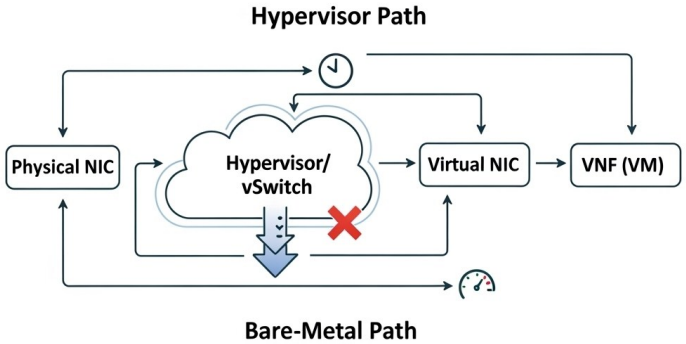
I/O Bottlenecks: The Biggest Challenge

- The most significant performance challenge in a virtualized networking environment lies in **packet I/O (Input/Output)**.
- Moving packets efficiently between the physical network interface card (NIC), the hypervisor, and the VNF's virtual NIC within a VM is a complex and resource-intensive operation. Traditional operating system network stacks and virtualization layers were not designed for the extreme packet rates required by modern network functions.

Hypervisor vs bare-metal network function

The big question is, how can we get software running on a generic CPU to process millions of packets per second, effectively bypassing these virtualization bottlenecks?

We need specialized techniques to achieve bare-metal-like performance in a virtual world.



Performance Optimization 1: Bypassing the Hypervisor (SR-IOV)

To overcome the I/O bottlenecks and achieve bare-metal-like performance for Virtualized Network Functions (VNFs), we employ specialized techniques. One powerful hardware-assisted method is **Single Root I/O Virtualization (SR-IOV)**.

SR-IOV (Single Root I/O Virtualization)

- SR-IOV is a feature of **modern Network Interface Cards (NICs)** and corresponding server hardware. It allows a single physical PCIe (Peripheral Component Interconnect Express) device, such as a physical NIC, to appear as multiple separate, lightweight virtual devices.

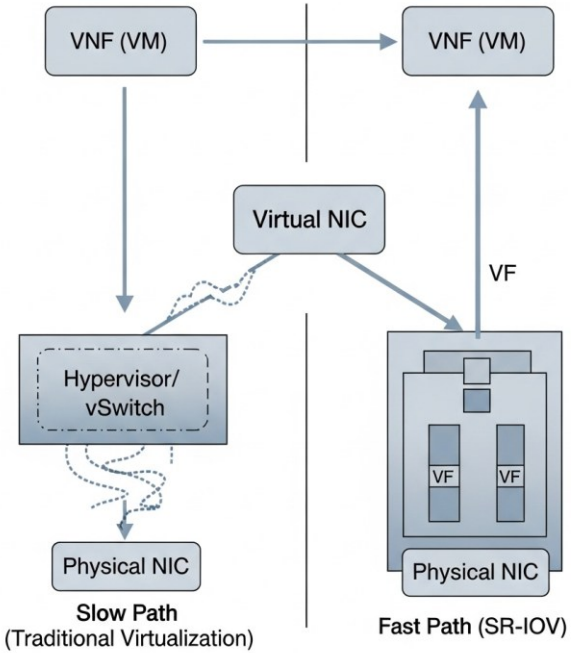
How it Works: Direct Access to Hardware

- **Virtual Functions (VFs):** SR-IOV enables a single physical NIC to expose multiple **Virtual Functions (VFs)**. Each VF behaves like a dedicated, independent network interface.
- **Direct Assignment:** A Virtual Machine (VM) or VNF Component (VNFC) can be given **direct, exclusive access** to one of these Virtual Functions. This is achieved by mapping the VF directly to the VM's memory space, bypassing the hypervisor's software-based network stack.
- **Bypassing the Hypervisor:** When a VM uses an SR-IOV VF, its network traffic no longer needs to traverse the hypervisor's virtual switch (vSwitch). Instead, packets flow directly between the VM and the physical NIC's VF.

Benefit: Near-Native Performance

- The primary benefit of SR-IOV is that it allows VNFs to achieve **near-native, low-latency I/O performance**.
- By completely **bypassing the hypervisor's virtual switch** and its associated overhead, SR-IOV significantly reduces latency and increases packet processing throughput, making it suitable for high-performance network functions like 5G UPFs.

SR-IOV (Single Root I/O Virtualization)



Performance Optimization 2: Bypassing the OS Kernel (DPDK)

While SR-IOV provides hardware-assisted bypass of the hypervisor, **DPDK (Data Plane Development Kit)** offers a powerful software-based approach to achieve high-speed packet processing by bypassing the traditional OS kernel network stack.

DPDK (Data Plane Development Kit)

- DPDK is a set of **open-source libraries** (primarily developed by Intel) designed to accelerate packet processing workloads running on general-purpose CPUs.

The Problem: The OS Kernel Network Stack

- The standard **Linux kernel network stack** is designed for general-purpose networking, prioritizing flexibility and broad compatibility over raw packet processing speed.
- It's inherently **slow for high-speed packet processing** due to several factors:
  - **Context Switches:** Frequent transitions between user space (application) and kernel space (OS) for every packet.
  - **Interrupts:** Each incoming packet generates an interrupt, consuming CPU cycles.
  - **Data Copies:** Data often needs to be copied multiple times between kernel and user memory.
  - These overheads make it inefficient for applications requiring millions of packets per second.

Performance Optimization 2: Bypassing the OS Kernel (DPDK)

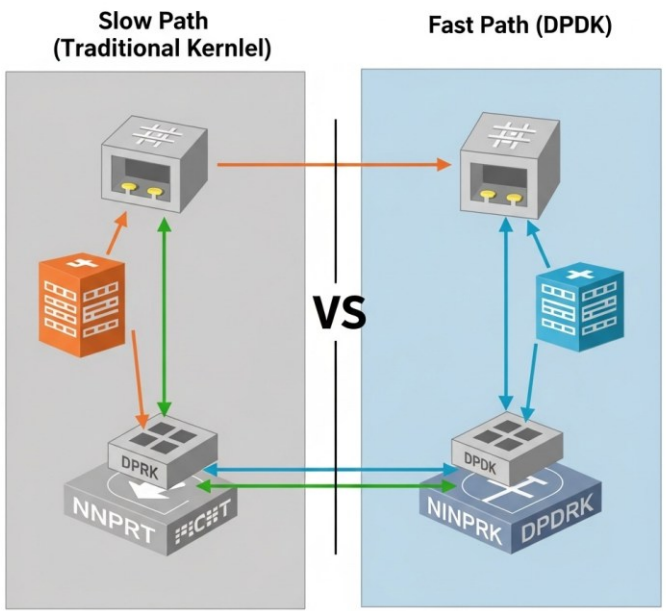
How it Works: Bypassing the Kernel

- DPDK allows a **user-space application (the VNF)** to take **direct control of the Network Interface Card (NIC) hardware**, effectively **bypassing the kernel entirely** for packet processing.

Impact: Enabling High-Performance NFV

- DPDK, often used in conjunction with SR-IOV, is one of the two fundamental technologies that truly **made high-performance NFV possible**.
- It allows VNFs to achieve **tens of millions of packets per second** on commodity x86 servers, making them suitable for demanding network functions like 5G User Plane Functions (UPFs), virtual routers, and virtual firewalls, which require wire-speed packet processing.

Slow Path vs Fast Path



The Next Generation: Cloud-Native Network Functions (CNFs) & Kubernetes

While NFV brought virtualization to network functions, the industry is now rapidly evolving towards **Cloud-Native Network Functions (CNFs)**, leveraging container technology and powerful orchestrators like Kubernetes. This represents the next leap in agility and efficiency.

From Monolithic VNFs to Microservices

- **Traditional VNFs (VM-based):** Often designed as large, somewhat monolithic applications running within a single Virtual Machine. While better than physical appliances, they still carried some VM overhead and could be slow to start or scale.
- **Cloud-Native Network Functions (CNFs):** A CNF is fundamentally different. It's designed as a set of **small, independent, containerized microservices**. Each microservice performs a specific, well-defined function (e.g., authentication, routing table management, session handling).
- **Benefits of Microservices:** This architectural shift allows for greater modularity, independent development and deployment of components, and improved fault isolation.

The Next Generation: Cloud-Native Network Functions (CNFs) & Kubernetes

Orchestration with Kubernetes (K8s)

- **Kubernetes (K8s)** has emerged as the de facto standard for orchestrating containerized applications. In the context of CNFs, Kubernetes effectively becomes the **new VIM/VNFM** (Virtualized Infrastructure Manager / VNF Manager).
- **K8s's Role:** It manages the entire lifecycle of these containers (often grouped into "Pods"), including:
  - **Deployment:** Placing containers on available compute nodes.
  - **Scaling:** Automatically increasing or decreasing the number of container instances based on load.
  - **Healing:** Detecting failed containers and automatically restarting or replacing them.
  - **Load Balancing:** Distributing traffic across multiple instances of a microservice.
  - **Resource Management:** Allocating CPU, memory, and network resources to containers.

The Next Generation: Cloud-Native Network Functions (CNFs) & Kubernetes

Key Cloud-Native Concepts

The shift to CNFs brings several crucial cloud-native principles to networking:

- **Statelessness:** Microservices are designed to be **stateless** wherever possible. This means they don't store session-specific data internally, making them easy to scale horizontally (just add more instances) and replace without losing user sessions. State is typically managed externally in a distributed database.
- **Immutability:** Once a container image is built, it's never changed. Updates involve deploying new container images, simplifying rollbacks and ensuring consistency.
- **CI/CD Pipelines (Continuous Integration/Continuous Delivery):** Automated building, testing, and deployment processes are fundamental. This enables rapid iteration and frequent, reliable updates to network functions.
- **Automation First:** Operations are driven by automation, reducing manual intervention and human error.

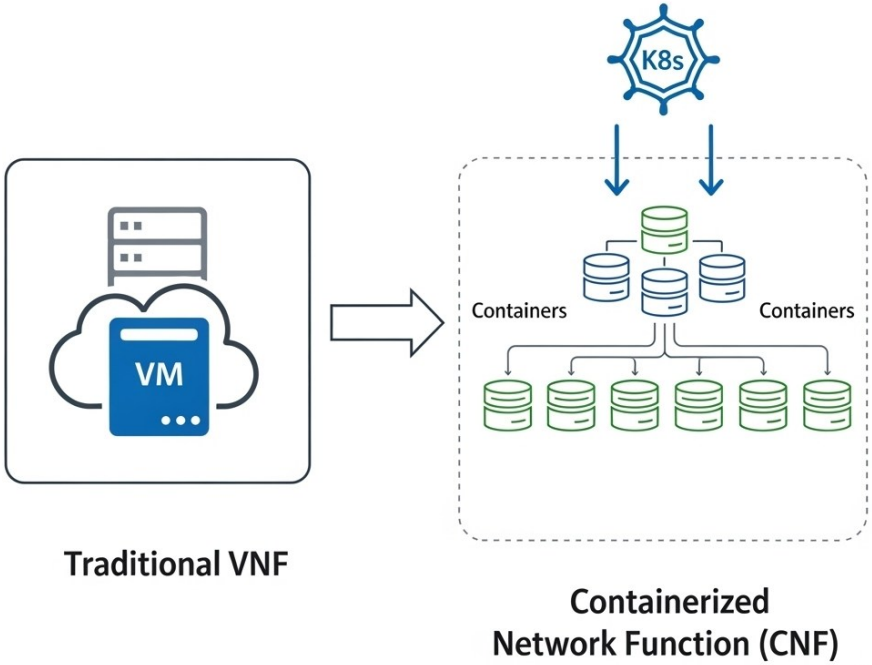
The Next Generation: Cloud-Native Network Functions (CNFs) & Kubernetes

Benefits: Unmatched Agility, Resilience, and Resource Efficiency

Embracing CNFs and Kubernetes delivers significant advantages:

- **Unmatched Agility:** Faster deployment times (seconds vs. minutes/hours for VMs), quicker updates, and rapid innovation cycles.
- **Enhanced Resilience:** Microservices architecture and Kubernetes' self-healing capabilities lead to more robust and fault-tolerant network functions.
- **Superior Resource Efficiency:** Containers are much lighter-weight than VMs, consuming fewer resources (CPU, RAM) per instance, leading to higher density and better hardware utilization.
- **Scalability:** The ability to scale individual microservices independently allows for more granular and efficient resource allocation.

Traditional VNF vs CNF



Key takeaways and Examples

NFV Summary: Key Concepts and Evolution

We've now explored Network Function Virtualization (NFV), a transformative paradigm that underpins modern and future networks.

Recap: NFV's Revolution

- **NFV revolutionizes network service delivery** by decoupling network functions from proprietary hardware.
- It enables these functions to run as **Virtualized Network Functions (VNFs)** on standard, commodity servers, bringing cloud-like agility to network operations.
- The entire system is managed and automated by the **NFV Management and Orchestration (MANO)** framework, which comprises the NFV Orchestrator (NFVO), VNF Manager (VNFM), and Virtualized Infrastructure Manager (VIM).

Key Takeaways: From Concept to Cloud-Native

Service Function Chaining (SFC):

- A critical use case enabled by NFV.
- Allows for the **dynamic creation of flexible, customized network service chains** by steering traffic through an ordered sequence of VNFs.
- This is where **SDN's programmable connectivity** plays a crucial role, enforcing the traffic steering between VNFs.

Performance Optimization:

- The "performance tax" of virtualization (hypervisor/vSwitch overhead) is a significant challenge for high-throughput network functions.
- This challenge is overcome by key optimization techniques:
  - **SR-IOV (Single Root I/O Virtualization):** A hardware-assisted method that allows VNFs to **bypass the hypervisor** for near-native I/O performance.
  - **DPDK (Data Plane Development Kit):** A software library that enables user-space applications (VNFs) to **bypass the OS kernel** for ultra-high-speed packet processing.
- These two technologies were instrumental in making high-performance NFV a reality.

The Future is Cloud-Native:

- The industry is rapidly moving from VM-based VNFs to **Cloud-Native Network Functions (CNFs).**
- CNFs are designed as **containerized microservices**, offering unmatched agility, resilience, and resource efficiency.
- **Kubernetes (K8s)** has become the de facto orchestrator for CNFs, effectively acting as the new VIM/VNFM for containerized network functions.
- Advanced concepts like **Service Meshes** (e.g., Istio, Linkerd) further enhance CNFs by transparently managing complex inter-service communication, security, and observability.

SDN and NFV: Complementary Technologies

While often discussed together, SDN and NFV are distinct technologies that, when combined, unlock the full potential of software-defined infrastructure.

Separate Pursuits, Shared Goals

- **Independent Development:** SDN and NFV have largely been developed by different standards bodies and industry groups, focusing on their respective areas of network control and function virtualization.
- **Industry Adoption:** Despite separate origins, many network providers and vendors are actively working on products and solutions in both fields, recognizing their synergistic value.

Potential for Added Value

- **Beyond Standalone Deployment:** While SDN and NFV can be deployed independently, their **coordinated use offers significant benefits** that go beyond what each technology can achieve alone.
- **Critical for 5G (and 6G):** This combined approach is especially valuable and increasingly prevalent in **5G core networks**, where the demands for flexibility, scalability, and efficiency are paramount.

SDN and NFV: Complementary Technologies

Interoperating Capability

- **Unified Software-Based Approach:** SDN and NFV are increasingly deployed together in modern networks, particularly in 5G. This provides a **unified software-based approach** to both control the underlying network equipment and manage virtualized network resources.
- **Dynamic Orchestration:** SDN can programmatically connect and manage the virtualized network functions (VNFs) provided by NFV, creating dynamic service chains and optimizing traffic flows.

Combined Benefits

The combined deployment of SDN and NFV significantly enhances network capabilities:

- **Enhanced Efficiency:** Optimizes resource utilization and reduces operational costs by running network functions on commodity hardware.
- **Increased Programmability:** Allows for rapid, automated configuration and management of the entire network infrastructure.
- **Greater Flexibility:** Enables dynamic scaling, rapid service deployment, and agile adaptation to changing network requirements.
- **Support for Diverse & Dynamic 5G Requirements:** Together, they provide the agility and scalability needed to support 5G's diverse use cases, from ultra-reliable low-latency communication to massive machine-type communications.

Application: The Virtualized 5G Core (vEPC/5GC)

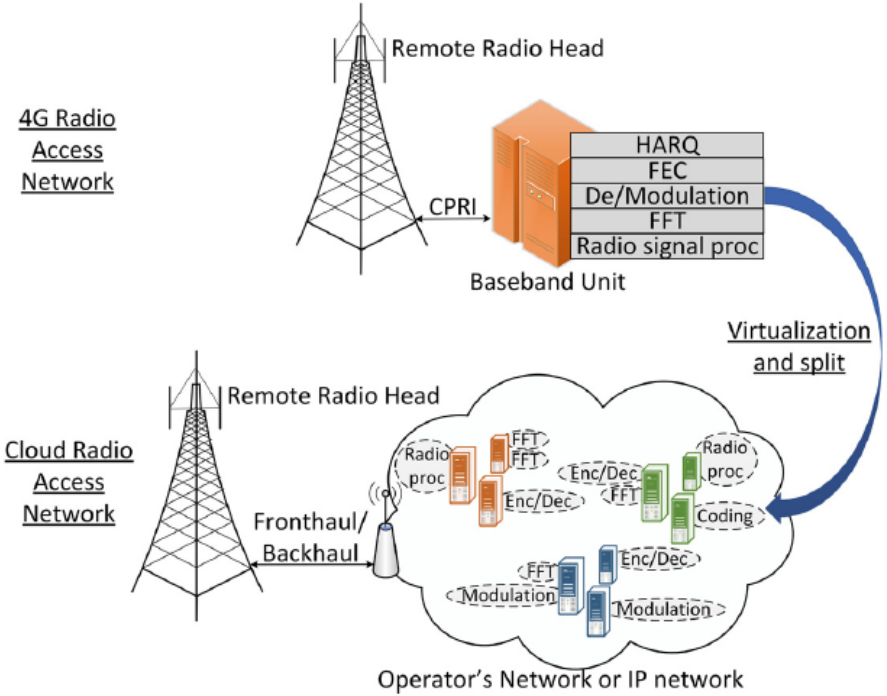
For decades, mobile network cores relied on dedicated hardware. With NFV, operators are now transforming their networks into a flexible, software-centric platform.

Key 5G Core functions, such as the **Access and Mobility Management Function (AMF)**, **Session Management Function (SMF)**, and **User Plane Function (UPF)**, are no longer rigid hardware boxes. They are deployed as **Virtual Network Functions (VNFs)** or **Cloud-Native Network Functions (CNFs)** running on standard servers in data centers.

Key Benefits

- **Cost Reduction (CapEx/OpEx):** Moving from expensive, single-purpose hardware to commodity servers significantly reduces both capital (CapEx) and operational (OpEx) costs.
- **Service Agility:** Virtualizing the core allows operators to launch new services rapidly. Functions can be deployed in minutes, not weeks or months.
- **Scalability:** The virtualized core is inherently elastic. Functions can automatically scale out or in to handle peak traffic, allowing the network to dynamically adapt to demand.

Virtualized 5G Core vs. Legacy 4G RAN



Relevance for YOU: What Can You Design?

To identify practical examples and design problems you can tackle using Network Functions Virtualization (NFV) and Software-Defined Networking (SDN) technologies.

By leveraging the flexibility of a software-defined infrastructure, there are many opportunities to innovate and solve complex problems across various domains.

Application Examples

- **Multimedia:** A **"transcoding as a service"** offering, where you dynamically instantiate VNFs to convert video formats based on the user's device. This ensures the best viewing experience on any screen, from mobile to desktop, without pre-processing every possible format.
- **Advanced Computing:** Optimization algorithms for **VNF Placement**: given a set of VNFs and an infrastructure, where should you place the VNFs to minimize latency or energy consumption? This is an **NP-hard** problem.
- **Big Data:** A security system that automatically scales by instantiating new traffic analysis VNFs when it detects a DDoS attack. The additional VNFs allow the system to handle the increased data volume and mitigate the attack in real time.

Numerical Example: Capacity Planning

To show a simple calculation for determining the maximum number of services that can be run on a server, identifying the resource that limits the capacity.

Problem:

A server has **64 vCPUs** and **256 GB of RAM**. A service requires a chain of 3 VNFs with the following specifications:

- **vFW**: 2 vCPUs, 4 GB RAM
- **vDPI**: 4 vCPUs, 8 GB RAM
- **vNAT**: 1 vCPU, 2 GB RAM

Question: What is the maximum number of service chains I can instantiate on this server? Which resource is the bottleneck (CPU or RAM)?

Numerical Example: Capacity Planning

Solution:

Calculate the total requirements for one service chain:

- **vCPUs:** 2+4+1=7 vCPUs
- **RAM:** 4+8+2=14 GB RAM

Calculate the maximum number of chains based on each resource:

- **Based on vCPUs:** $64 \div 7 \approx 9.14 \rightarrow 9$ chains
- **Based on RAM:** $256 \div 14 \approx 18.28 \rightarrow 18$ chains

Identify the bottleneck: The maximum number of chains we can instantiate is the lowest value between the two calculations. In this case, the **bottleneck** is the **CPU**, since it only allows for **9 chains** to be instantiated.

Answer: A maximum of **9 service chains** can be instantiated. The resource acting as the bottleneck is the **CPU**.

UNIVERSITÀ DEGLI STUDI FIRENZE
Da un secolo, oltre.

HR EXCELLENCE IN RESEARCH

02.2: Network Function Virtualization (NFV)

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi
Dipartimento Ingegneria dell'Informazione
daniele.tarchi@unifi.it